

存储管理文档

任务一、缓冲池管理器

测试单元

任务1.1 磁盘存储管理器

基本实现

基本实现

基本实现

基本实现

基本实现

任务1.2 缓冲池替换策略

基本实现

基本实现

任务1.3 缓冲池管理器

基本实现

基本实现

基本实现

基本实现

缓冲池管理器的几个重要场景

- 1、通过页头信息(PageId)找到页指针地址

任务二 记录管理器

任务2.1 记录操作

任务2.2 记录迭代器

实验计分

任务一、缓冲池管理器

数据在磁盘文件中是按照页面（Page）形式组织的。为避免直接访问磁盘数据页面而造成高昂的I/O开销，存储子系统在内存中创建缓冲池（Buffer Pool）来缓存部分磁盘数据页面。缓冲池维护固定数量的内存页面，每个内存页面称为“帧”（Frame），一般情况下，每一帧的大小与磁盘数据页面的大小保持一致。受制于内存容量，缓冲池只能缓存部分数据页面。因此，缓冲池管理的目

标，就是在受限缓冲池大小的前提下，设计合适的内外存页面调度策略，尽可能将经常访问的磁盘数据页面维护在缓冲池中，从而减少磁盘I/O开销。

本实验涉及缓冲池管理的重要内容，学生要实现数据库存储系统中的缓冲池管理器，即BufferPoolManager类。它的数据结构包括Page、DiskManager、Replacer类的对象等。其中，Page类已提供，不需要实现，其路径位于src/storage/page.h。

本实验要求学生依次完成三个子任务，实现DiskManager、Replacer、BufferPoolManager类的接口。

测试单元

disk_manager_test

lru_replacer_test

buffer_pool_manager_test

record_manager_test

任务1.1 磁盘存储管理器

本任务要求补全DiskManager类，其负责读写指定页面、分配页面编号，以及对文件进行操作。

DiskManager类的接口如下：

```
▼ DiskManager C++
1 class DiskManager {
2 public:
3     explicit DiskManager();           // Constructor
4     ~DiskManager() = default;        // Destructor
5     // 读写页面
6     void write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);
7     void read_page(int fd, page_id_t page_no, char *offset, int num_bytes);
8     // 对指定文件分配页面编号
9     page_id_t allocate_page(int fd);
10    // 文件操作
11    bool is_file(const std::string &path);
12    void create_file(const std::string &path);
13    int open_file(const std::string &path);
14    void close_file(int fd);
15    void destroy_file(const std::string &path);
16 }
```

这些接口的内部实现调用了Linux操作系统下/usr/include/unistd.h提供的read()、write()、open()、close()，unlink()等函数。

(1) 读写页面

- void write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);
- void read_page(int fd, page_id_t page_no, char *offset, int num_bytes);

提示：可以调用read()或write()函数。通过(fd,page_no)可以定位指定页面及其在磁盘文件中的偏移量。注意：这里支持读写的字节长度为num_bytes，上层调用此函数读写页面时，其值一般为页面大小PAGE_SIZE。但有时也可以小于PAGE_SIZE，比如只读写页头数据。

基本实现

write_page

C++

```
1 void DiskManager::write_page(int fd, page_id_t page_no, const char *offset,
  int num_bytes) {
2     lseek(fd, page_no * PAGE_SIZE, SEEK_SET);    // 定位到第page_no页
3     write(fd, offset, num_bytes);
4     // Todo:
5     // 1.lseek()定位到文件头，通过(fd,page_no)可以定位指定页面及其在磁盘文件中的偏移
    量
6     // 2.调用write()函数
7     // 注意write返回值与num_bytes不等时 throw InternalError("DiskManager::write_page Error");
8
9 }
```

read_page

C++

```
1 void DiskManager::read_page(int fd, page_id_t page_no, char *offset, int num_bytes) {
2     lseek(fd, page_no * PAGE_SIZE, SEEK_SET);    // 定位到第page_no页
3     read(fd, offset, num_bytes);
4     // Todo:
5     // 1.lseek()定位到文件头，通过(fd,page_no)可以定位指定页面及其在磁盘文件中的偏移
    量
6     // 2.调用read()函数
7     // 注意read返回值与num_bytes不等时, throw InternalError("DiskManager::read_page Error");
8
9 }
10
```

(2) 分配页面编号

- `page_id_t allocate_page(int fd);`

```
▼ allocate_page C++ |
1  page_id_t DiskManager::allocate_page(int fd) {
2      // 简单的自增分配策略，指定文件的页面编号加1
3      assert(fd >= 0 && fd < MAX_FD);
4      return fd2pageno_[fd]++;
5  }
```

目前采取简单的自增分配策略：指定文件的页面编号加1。

(3) 文件操作

- `bool is_file(const std::string &path);`

```
▼ is_file C++ |
1  bool DiskManager::is_file(const std::string &path) {
2      // 用struct stat获取文件信息
3      struct stat st;
4      return stat(path.c_str(), &st) == 0 && S_ISREG(st.st_mode);
5  }
```

用于判断指定路径文件是否存在。

提示：用struct stat获取文件信息。

- `void create_file(const std::string &path);`

基本实现

▼ create_file

C++

```
1 void DiskManager::create_file(const std::string &path) {
2     if (this->is_file(path)) {
3         throw FileExistsError(path);
4     }
5     int fd = open(path.c_str(), O_RDWR | O_CREAT, 0644);
6     if (fd == -1) {
7         throw UnixError();
8     }
9     close(fd);
10
11     // Todo:
12     // 调用open()函数, 使用O_CREAT模式
13     // 注意不能重复创建相同文件
14 }
```

用于创建指定路径文件。

提示：调用open()函数，使用O_CREAT模式。注意不能重复创建相同文件。

- void open_file(const std::string &path);

基本实现

▼ open_file

C++

```
1 int DiskManager::open_file(const std::string &path) {
2     if (!this->is_file(path)) {
3         // 文件不存在, 抛出 FileNotFoundError
4         throw FileNotClosdError(path);
5     }
6     if (this->path2fd_.count(path)) { // 已打开则不重复打开
7         throw FileNotClosdError(path);
8     }
9     int fd = open(path.c_str(), O_RDWR);
10    if (fd < 0) {
11        throw UnixError();
12    }
13    return fd;
14    // Todo:
15    // 调用open()函数, 使用O_RDWR模式
16    // 注意不能重复打开相同文件, 并且需要更新文件打开列表
17
18 }
```

用于打开指定路径文件。

提示：调用open()函数，使用O_RDWR模式。注意不能重复打开相同文件，并且需要更新文件打开列表。

- void close_file(int fd);

基本实现

```
close_file C++
1 void DiskManager::close_file(int fd) {
2     if(!this->fd2path_.count(fd)) { // 未打开则不关闭
3         throw FileNotOpenError(fd);
4         return;
5     }
6     if(close(fd) == -1) {
7         throw UnixError();
8         return;
9     }
10    // Todo:
11    // 调用close()函数
12    // 注意不能关闭未打开的文件，并且需要更新文件打开列表
13
14 }
```

用于关闭指定路径文件。

提示：调用close()函数。注意不能关闭未打开的文件，并且需要更新文件打开列表。

- void destroy_file(const std::string &path);

基本实现

▼ destroy_file

C++

```
1 void DiskManager::destroy_file(const std::string &path) {
2     if(!this->is_file(path)) { //是否存在
3         throw FileNotFoundError(path);
4     }
5     if(this->path2fd_.count(path)) { //是否未关闭
6         throw FileNotClosedError(path);
7     }
8     if(unlink(path.c_str()) < 0) {
9         throw UnixError();
10    }
11    // Todo:
12    // 调用unlink()函数
13    // 注意不能删除未关闭的文件
14
15 }
```

用于删除指定路径文件。

提示：调用unlink()函数。注意不能删除未关闭的文件。

任务1.2 缓冲池替换策略

本任务要求补全 `Replacer` 类，其负责跟踪缓冲池中每个页面所在帧的使用情况。当缓冲池没有空闲页面时，需要使用该类提供的替换策略选择一个页面进行淘汰。要求实现的替换策略为最近最少使用（LRU）算法。

LRU算法是通过一个双向链表加一个哈希表的数据结构来实现的。

`Replacer` 类的接口如下：

```
▼ C++
1 class Replacer {
2 public:
3     explicit Replacer(size_t num_pages);
4     ~Replacer();
5     bool victim(frame_id_t *frame_id);
6     void pin(frame_id_t frame_id);
7     void unpin(frame_id_t frame_id);
8 };
```

注意需要保证每个函数都是原子性的操作，可以使用 `std::mutex` 对每个函数上锁。

这里的优化策略可以考虑将互斥锁替换成自旋锁。

- `Replacer(size_t num_pages);`

▼ LRUReplacer

C++

```
1 LRUReplacer::LRUReplacer(size_t num_pages) { max_size_ = num_pages; }
```

构造函数，初始化 `LRUlist` 的最大容量 `max_size`。

- `bool victim(frame_id_t *frame_id);`

基本实现

▼ victim

C++

```
1 bool LRUReplacer::victim(frame_id_t* frame_id) {
2     // C++17 std::scoped_lock
3     // 它能够避免死锁发生，其构造函数能够自动进行上锁操作，析构函数会对互斥量进行解锁操作，保证线程安全。
4     std::scoped_lock lock{latch_}; // 如果编译报错可以替换成其他lock
5
6
7     if(this->LRUlist_.empty()){
8         frame_id = nullptr;
9         return false;
10    }
11    else{
12        *frame_id = this->LRUlist_.back();
13        this->LRUlist_.pop_back();
14        this->LRUhash_.erase(*frame_id);
15        return true;
16    }
17
18    // Todo:
19    // 利用lru_replacer中的LRUlist_,LRUHash_实现LRU策略
20    // 选择合适的frame指定为淘汰页面,赋值给*frame_id
21
22 }
```

当缓冲池要淘汰一个页面所在的帧，调用此函数。这里对实现的方法做适当的说明

- 当LUR链为空时，说明没有任何数据会被淘汰，将帧指针设置为空并返回错误。
- 当实现淘汰时，返回LRU链的最后一个数据帧，从LRU链中删除该数据同时也要在哈希表中删除该数据。哈希表的作用是快速查找页面的 ID 是否存在于缓存(LRU链)中。

需要删除 `LRUlist` 中最远被unpin的帧，并传出该帧的编号。

- `void pin(frame_id_t frame_id);`

```
pin
C++

1 void LRUReplacer::pin(frame_id_t frame_id) {
2     std::scoped_lock lock{latch_};
3
4     auto find = this->LRUhash_.find(frame_id);
5     if(find != this->LRUhash_.end()){
6         this->LRUlist_.erase(find->second);
7         this->LRUhash_.erase(find);
8     }
9     // Todo:
10    // 固定指定id的frame
11    // 在数据结构中移除该frame
12 }
13
```

当缓冲池要固定一个页面所在的帧，调用此函数。这里对实现的方法做适当的说明

- LRUhash_.find是一个迭代器，如果找到了对应的帧id，则返回该指针，如果没找到则返回LRUhash_.end()
- 在哈希表中找到对应的帧id时，删除链表里对应的数据。
- 删除哈希表中的对应数据。

需要删除 LRUlist 中指定的帧，若该帧不存在则无任何操作。

- `void unpin(frame_id_t frame_id);`

基本实现

```
1 void LRUReplacer::unpin(frame_id_t frame_id) {
2
3     std::scoped_lock lock{latch_};
4
5     auto find = LRUhash_.find(frame_id);
6     if(find == LRUhash_.end()) {
7         LRUlist_.push_front(frame_id);
8         LRUhash_[frame_id] = LRUlist_.begin();
9     }
10    // Todo:
11    // 支持并发锁
12    // 选择一个frame取消固定
13 }
```

当缓冲池要取消固定一个页面所在的帧（该页面的 `pin_count` 变为0），调用此函数。这里对实现的方法做适当说明

- `LRUhash_.find`是一个迭代器，如果找到了对应的帧id，则返回该指针，如果没找到则返回 `LRUhash_.end()`
- 在哈希表中未找到对应的帧id时，将要加入淘汰算法的帧id加入LRU链头部。
- 在哈希表中加入帧id方便索引。

需要将指定帧插入到 `LRUlist` 中最近被unpin的位置。

任务1.3 缓冲池管理器

本任务要求补全 `BufferPoolManager` 类，其负责管理缓冲池中的页面与磁盘文件中的页面之间的来回移动。

`BufferPoolManager` 类的接口如下：

```
1 class BufferPoolManager {
2 public:
3     BufferPoolManager(size_t pool_size, DiskManager *disk_manager);
4     ~BufferPoolManager();
5     Page *new_page(PageId *page_id);
6     Page *fetch_page(PageId page_id);
7     bool unpin_page(PageId page_id, bool is_dirty);
8     bool delete_page(PageId page_id);
9     bool flush_page(PageId page_id);
10    void flush_all_pages(int fd);
11 private:
12    // 辅助函数
13    bool find_victim_page(frame_id_t *frame_id);
14    void update_page(Page *page, PageId new_page_id, frame_id_t new_frame_id);
15 }
```

注意要对缓冲池进行并发控制，可以像 `Replacer` 类那样直接用 `std::mutex` 对每个函数上锁。但这种方法实际上是顺序执行若干个原子性操作。学生可以考虑这些函数之间的并发执行逻辑，加以改进。

学生可以自主添加私有辅助函数，将某些重复使用的逻辑模块化，例如寻找淘汰页、更新页表和页面等。但是不允许修改任何公有函数的声明。

首先，可以实现辅助函数：

- `bool find_victim_page(frame_id_t *frame_id);`

基本实现

```
1  bool BufferPoolManager::find_victim_page(frame_id_t* frame_id) {
2
3      if(this -> free_list_.empty()){
4          if(this -> replacer_->victim(frame_id)){
5              return false;
6          }
7      }else{
8          *frame_id = this->free_list_.front();// 还有空闲帧,直接使用
9          this->free_list_.pop_front();
10     }
11     return true;
12
13     // Todo:
14     // 1 使用BufferPoolManager::free_list_判断缓冲池是否已满需要淘汰页面
15     // 1.1 未满足获得frame
16     // 1.2 已满使用lru_replacer中的方法选择淘汰页面
17 }
```

用于寻找淘汰页。

- `void update_page(Page *page, PageId new_page_id, frame_id_t new_frame_id);`

```

1 void BufferPoolManager::update_page(Page *page, PageId new_page_id, frame_
  id_t new_frame_id) {
2     if(page->is_dirty()){
3         this->disk_manager_->write_page(page->get_page_id().fd, page->get_
  page_id().page_no, page->get_data(), PAGE_SIZE);
4         page->is_dirty_ = false;
5     }
6
7     // 擦除页面中的数据
8     page->reset_memory();
9
10    // 从页表中擦除数据
11    for(auto position = this->page_table_.begin(); position != this->page_
  table_.end(); position++) { //更新table
12        if(position->first == page->id_) {
13            this->page_table_.erase(position);
14            break;
15        }
16    }
17
18    this->page_table_[new_page_id] = new_frame_id;
19
20    page->id_ = new_page_id;
21    if(page->id_.page_no != INVALID_PAGE_ID) {
22        this->disk_manager_->read_page(page->get_page_id().fd, page->get_p
  age_id().page_no, page->get_data(), PAGE_SIZE);
23    }
24    // Todo:
25    // 1 如果是脏页, 写回磁盘, 并且把dirty置为false
26    // 2 更新page table
27    // 3 重置page的数据, 更新page id
28
29 }

```

用于更新页表和页面。

update_page是缓冲池管理器中较为重要的函数, 其中包括将缓冲池中的数据写入磁盘以及将磁盘中的数据读入缓冲器的过程。

然后实现public函数:

- `BufferPoolManager(size_t pool_size, DiskManager *disk_manager);`

```

1  class BufferPoolManager {
2      private:
3          size_t pool_size_;          // buffer_pool中可容纳页面的个数，即帧的个数
4          Page *pages_;              // buffer_pool中的Page对象数组，在构造空间中申请内
// 存空间，在析构函数中释放，大小为BUFFER_POOL_SIZE
5          std::unordered_map<PageId, frame_id_t, PageIdHash> page_table_; // 帧
// 号和页面号的映射哈希表，用于根据页面的PageId定位该页面的帧编号
6          std::list<frame_id_t> free_list_;    // 空闲帧编号的链表
7          DiskManager *disk_manager_;
8          Replacer *replacer_;    // buffer_pool的置换策略，当前赛题中为LRU置换策略
9          std::mutex latch_;      // 用于共享数据结构的并发控制
10
11     public:
12         BufferPoolManager(size_t pool_size, DiskManager *disk_manager)
13             : pool_size_(pool_size), disk_manager_(disk_manager) {
14             // 为buffer pool分配一块连续的内存空间
15             pages_ = new Page[pool_size_];
16             // 可以被Replacer改变
17             if (REPLACER_TYPE.compare("LRU"))
18                 replacer_ = new LRUReplacer(pool_size_);
19             else if (REPLACER_TYPE.compare("CLOCK"))
20                 replacer_ = new LRUReplacer(pool_size_);
21             else {
22                 replacer_ = new LRUReplacer(pool_size_);
23             }
24             // 初始化时，所有的page都在free_list_中
25             for (size_t i = 0; i < pool_size_; ++i) {
26                 free_list_.emplace_back(static_cast<frame_id_t>(i)); // stati
// c_cast转换数据类型
27             }
28         }
29
30         ~BufferPoolManager() {
31             delete[] pages_;
32             delete replacer_;
33         }
34
35         /**
36          * @description: 将目标页面标记为脏页
37          * @param {Page*} page 脏页
38          */
39         static void mark_dirty(Page* page) { page->is_dirty_ = true; }
40
41     public:
42         Page* fetch_page(PageId page_id);

```

```

43
44     bool unpin_page(PageId page_id, bool is_dirty);
45
46     bool flush_page(PageId page_id);
47
48     Page* new_page(PageId* page_id);
49
50     bool delete_page(PageId page_id);
51
52     void flush_all_pages(int fd);
53
54 private:
55     bool find_victim_page(frame_id_t* frame_id);
56
57     void update_page(Page* page, PageId new_page_id, frame_id_t new_frame_
58 id);
    };

```

构造函数，需要初始化缓冲池的最大容量 `pool_size`，以及分配 `replacer` 和 `pages` 的地址空间。

初始时 `free_list` 中帧编号的范围为`[0,pool_size)`。

- `Page *new_page(PageId *page_id);`

基本实现

```

1 Page* BufferPoolManager::new_page(PageId* page_id) {
2
3     std::scoped_lock lock{latch_};
4
5     frame_id_t id;
6     if(this->find_victim_page(&id)) { //找到一个位置
7         page_id->page_no = this->disk_manager->allocate_page(page_id->fd)
; //获取编号
8         this->update_page(&this->pages_[id], *page_id, id); //更新page
9         this->replacer->pin(id);
10        this->pages_[id].pin_count_ = 1;
11
12    } else {
13        return nullptr;
14    }
15    return &this->pages_[id];
16    // 1. 获得一个可用的frame, 若无法获得则返回nullptr
17    // 2. 在fd对应的文件分配一个新的page_id
18    // 3. 将frame的数据写回磁盘
19    // 4. 固定frame, 更新pin_count_
20    // 5. 返回获得的page
21
22 }

```

用于在内存申请创建一个新的页面。

内部实现逻辑包括更新页表和页面、固定页面、寻找淘汰页等。

此外，需要用 `DiskManager` 分配页面编号，并传出这个新页面的编号。

提示：需要用到之前实现的接口 `Replacer::pin()`、`Replacer::victim()`、`DiskManager::allocate_page()` 等。

- `Page *fetch_page(PageId page_id);`

基本实现

```

1  Page* BufferPoolManager::fetch_page(PageId page_id) {
2      frame_id_t frame_id = INVALID_FRAME_ID;
3      bool flag = false;
4
5      std::scoped_lock lock{latch_};
6
7      if(this->page_table_.find(page_id) != this->page_table_.end()){
8          frame_id = this->page_table_[page_id];
9          flag = true;
10     }
11     else{
12         if(!this->find_victim_page(&frame_id)){
13             return nullptr;
14         }
15         this->update_page(&this->pages_[frame_id], page_id, frame_id);
16     }
17
18     this->replacer_->pin(frame_id);
19     if (flag){
20         this->pages_[frame_id].pin_count++;
21     }
22     else{
23         this->pages_[frame_id].pin_count=1;
24     }
25
26     return &this->pages_[frame_id]; //返回时自动解锁
27
28     //Todo:
29     // 1.      从page_table_中搜寻目标页
30     // 1.1     若目标页有被page_table_记录, 则将其所在frame固定(pin), 并返回目标
31     页。
32     // 1.2     否则, 尝试调用find_victim_page获得一个可用的frame, 若失败则返回null
33     ptr
34     // 2.      若获得的可用frame存储的为dirty page, 则须调用update_page将page写回
35     到磁盘
36     // 3.      调用disk_manager_的read_page读取目标页到frame
37     // 4.      固定目标页, 更新pin_count_
38     // 5.      返回目标页
39 }

```

用于获取缓冲池中的指定页面。

内部实现逻辑包括更新页表和页面、固定页面、寻找淘汰页等。

此外，如果缓冲池中不存在该页面，需要用 `DiskManager` 从磁盘中读取。

提示：需要用到之前实现的接口 `Replacer::pin()`、`Replacer::victim()`、`DiskManager::read_page()` 等。

- `bool unpin_page(PageId page_id, bool is_dirty);`

基本实现

```
▼ unpin_page C++ |
1 bool BufferPoolManager::unpin_page(PageId page_id, bool is_dirty) {
2
3     std::scoped_lock lock{latch_};
4     if(this->page_table_.find(page_id) == this->page_table_.end()) { //不
        存在
5         return false;
6     }
7     frame_id_t id = this->page_table_[page_id]; //获取id
8     Page* page = &this->pages_[id]; //通过id获取page
9
10    if(page->pin_count_ <= 0) {
11        return false;
12    }
13    else{
14        page->pin_count_-=1;
15    }
16    if(page->pin_count_ == 0) {
17        this->replacer_->unpin(id);
18    }
19    if (is_dirty){
20        page->is_dirty_ = is_dirty;
21    }
22    return true;
23
24    // Todo:
25    // 0. lock latch
26    // 1. 尝试在page_table_中搜寻page_id对应的页P
27    // 1.1 P在页表中不存在 return false
28    // 1.2 P在页表中存在，获取其pin_count_
29    // 2.1 若pin_count_已经等于0，则返回false
30    // 2.2 若pin_count_大于0，则pin_count_自减一
31    // 2.2.1 若自减后等于0，则调用replacer_的Unpin
32    // 3 根据参数is_dirty，更改P的is_dirty_
33 }
```

用于使用完页面后，对该页面取消固定。

内部实现逻辑较简单，先减少页面的一次引用次数，由于页面可能同时被多个线程使用，调用一次 `unpin_page()` 只会减少一次引用次数，只有当引用次数减少到0时，才能调用 `Replacer::unpin()` 来取消固定页面所在的帧。

参数 `is_dirty` 决定是否对页面置脏，如果上层修改了页面，就将该页面的脏标志置 `true`。

- `bool delete_page(PageId page_id);`

```
▼ delete_page C++ |
1  bool BufferPoolManager::delete_page(PageId page_id) {
2      // 1. 在page_table_中查找目标页，若不存在返回true
3      // 2. 若目标页的pin_count不为0，则返回false
4      // 3. 将目标页数据写回磁盘，从页表中删除目标页，重置其元数据，将其加入free_list_，返回true
5
6      std::scoped_lock lock{latch_};
7
8      if(this->page_table_.find(page_id) == this->page_table_.end()) {
9          return true;
10     }
11     frame_id_t id = this->page_table_[page_id];
12     Page* page = &this->pages_[id];
13     if(page->pin_count_ != 0) { //还在被使用，不能删除
14         return false;
15     }
16     this->disk_manager_>deallocate_page(page->get_page_id().page_no);
17     page_id.page_no = INVALID_PAGE_ID;
18     this->update_page(page, page_id, id); //包含page table处理
19     this->free_list_.push_back(id);
20     return true;
21 }
```

用于删除指定页面。

内部实现逻辑包括更新页表和页面、更新空闲帧列表等。

注意：只有引用次数为0的页面才能被删除。

- `bool flush_page(PageId page_id);`

```
1 bool BufferPoolManager::flush_page(PageId page_id) {
2
3     std::scoped_lock lock{latch_};
4
5     if(this->page_table_.find(page_id) == this->page_table_.end()) {
6         return false;
7     }
8     frame_id_t id = this->page_table_[page_id]; //获取id
9     Page* page = &this->pages_[id]; //通过id获取page
10
11     this->disk_manager_->write_page(page->get_page_id().fd, page->get_page
    _id().page_no, page->get_data(), PAGE_SIZE);
12     // page->set_dirty(false);
13     page->is_dirty_ = false;
14     return true;
15
16     // Todo:
17     // 0. lock latch
18     // 1. 查找页表,尝试获取目标页P
19     // 1.1 目标页P没有被page_table_记录 , 返回false
20     // 2. 无论P是否为脏都将其写回磁盘。
21     // 3. 更新P的is_dirty_
22
23 }
```

用于强制刷新（写入）缓冲池中的指定页面到磁盘。

此处的"强制"指的是无论该页的引用次数是否大于0，无论该页是否为脏页，都将其刷新到磁盘。

- `void flush_all_pages(int fd);`

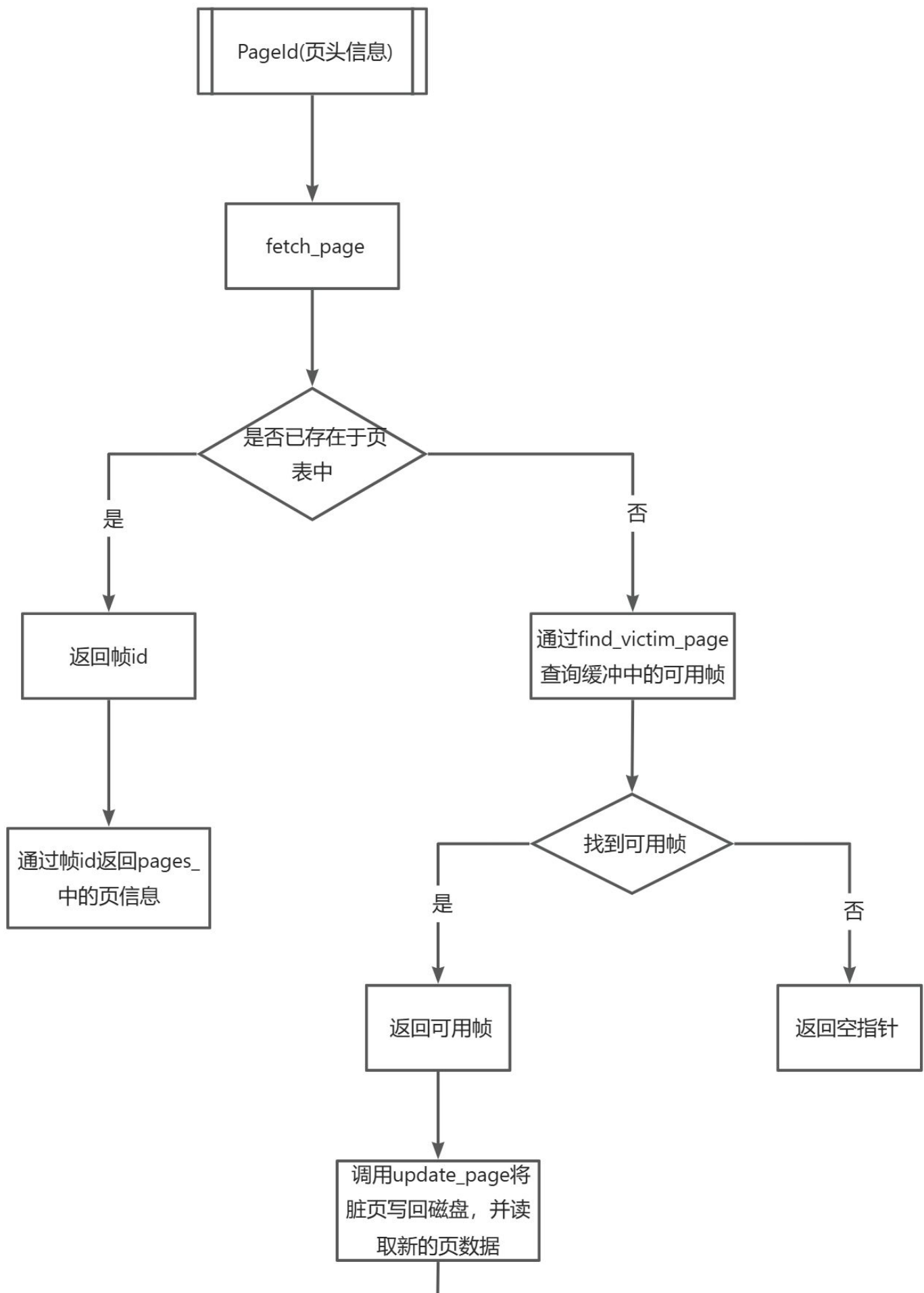
```
1 void BufferPoolManager::flush_all_pages(int fd) {
2     std::scoped_lock lock{latch_};
3     for (size_t i = 0; i < pool_size_; i++) {
4         Page *page = &this->pages_[i];
5         if (page->get_page_id().fd == fd && page->get_page_id().page_no !=
INVALID_PAGE_ID) {
6             disk_manager_->write_page(page->get_page_id().fd, page->get_page_id().page_no, page->get_data(), PAGE_SIZE);
7             page->is_dirty_ = false;
8         }
9     }
10
11 }
```

用于将指定文件中的存在于缓冲池的所有页面都刷新到磁盘。

注意，在上述所有函数的实现中，淘汰脏页之前，都要将脏页写入磁盘。

缓冲池管理器的几个重要场景

1、通过页头信息(PageId)找到页指针地址





任务二 记录管理器

数据库表中的一行数据，称为元组（Tuple）或者记录（Record），每条记录由多个字段（Field）组成。记录本质上就是一个字节序列，DBMS存储系统负责将其解释成属性类型和值。记录虽然是存放在磁盘而不是内存中，但是对记录的操作仍需在内存中进行，所以在组织记录时需要考虑如何让它在内存能够被高效访问。

根据字段的长度是否固定，可以分为定长字段和变长字段。根据记录是否存在变长字段，可以分为定长记录和变长记录。定长记录全部由定长字段组成，是比较简单的记录组织形式，只需要存储实际数据和固定的字段长度。本实验采用定长记录的组织形式，便于计算某个定长字段在记录中的偏移位置。

在本实验中，学生需要实现存储系统中的记录管理器，它主要由 `RMManager` 类、`RMFileHandle` 类、`RMPageHandle` 类、`RMScan` 类组成。此外，还有底层数据结构的 `Rid` 类和 `RmReocrd` 类。

其中，学生只要实现 `RMFileHandle` 和 `RMScan` 类中的接口。已提供其他类的完整源码。

`RMManager` 类提供了创建/打开/关闭/删除记录文件的接口，其内部实现调用了任务一实现的 `DiskManager` 和 `BufferPoolManager` 类的接口。

`RMPageHandle` 类的介绍参见项目结构文档。

任务2.1 记录操作

本任务要求补全 `RMFileHandle` 类，其负责对文件中的记录进行操作。

每个 `RMFileHandle` 对应一个记录文件，当 `RMManager` 执行打开文件操作时，便会创建一个指向 `RMFileHandle` 的指针。

`RMFileHandle` 类的接口如下：

```

1 class RmFileHandle {
2     public:
3         RmFileHandle(DiskManager *disk_manager, BufferPoolManager *buffer_pool
4             _manager, int fd);
5         // 不考虑事务的记录操作（事务将在后续实验使用）
6         std::unique_ptr<RmRecord> get_record(const Rid &rid, Context *context)
7             const;
8         Rid insert_record(char *buf, Context *context);
9         void delete_record(const Rid &rid, Context *context);
10        void update_record(const Rid &rid, char *buf, Context *context);
11        // 辅助函数
12        RmPageHandle create_new_page_handle();
13        RmPageHandle fetch_page_handle(int page_no) const;
14        RmPageHandle create_page_handle();
15        void release_page_handle(RmPageHandle &page_handle);
16    };

```

首先，需要实现辅助函数：

(1) `RmPageHandle create_new_page_handle();`

用于创建一个新的 `RmPageHandle`。

用缓冲池创建新页，并更新 `page_hdr` 和 `file_hdr` 中的各项内容。

提示：调用 `BufferPoolManager::new_page()` 创建新页面。

(2) `RmPageHandle fetch_page_handle(int page_no) const;`

用于获取指定页面对应的 `RmPageHandle`。

提示：调用 `BufferPoolManager::fetch_page()` 获取指定页面。

(3) `RmPageHandle create_page_handle();`

用于创建或获取一个空闲的 `RmPageHandle`。

内部实现逻辑是先判断第一个空闲页是否存在，如果存在就直接用 `fetch_page_handle()` 获取它；否则直接用 `create_new_page_handle()` 创建一个新的 `RmPageHandle`。

(4) `void release_page_handle(RmPageHandle &page_handle);`

当page handle中的page从已满变成未满的时候调用此函数。

提示：更新 `page_hdr` 的下一个空闲页和 `file_hdr` 的第一个空闲页。

然后，可以实现其他的public函数：

```
(5) RmFileHandle(DiskManager *disk_manager, BufferPoolManager *buffer_pool_manager, int fd);
```

构造函数，当上层创建一个指向 `RmFileHandle` 的指针时将会在此初始化。需要从磁盘中读出文件头的信息，即初始化 `file_hdr`。还需要设置该文件中开始分配的页面编号。

```
(6) std::unique_ptr<RmRecord> get_record(const Rid &rid, Context *context) const;
```

用于获取一条指定记录。由Rid得到record。

内部实现逻辑是把位于指定slot的记录拷贝一份，然后返回给上层。

```
(7) Rid insert_record(char *buf, Context *context);
```

用于插入一条指定记录。

对于堆文件组织形式，只需要找到一个有足够空间存放该记录的页面。当所有已分配页面中都没有空间时，就申请一个新页面来存放该记录。

注意更新bitmap，它跟踪了每个slot是否存放了record；此外，如果当前page handle中的page插入后已满，还需要更新 `file_hdr` 的第一个空闲页。

```
(8) void delete_record(const Rid &rid, Context *context);
```

用于删除一条指定记录。使用rid中的page_no和slot_no。

先获取page handle，然后将page的bitmap中表示对应槽位的bit置0。

注意如果删除操作导致该页面恰好从已满变为未满，那么需要调用 `release_page_handle()`。

```
(9) void update_record(const Rid &rid, char *buf, Context *context);
```

用于更新一条指定记录。

先获取page handle，然后直接更新page即可。

任务2.2 记录迭代器

本任务要求补全 `RmScan` 类，其用于遍历文件中存放的记录。

`RmScan` 类继承于 `RecScan` 类，它们的接口如下：

```
1 class RecScan {
2 public:
3     virtual ~RecScan() = default;
4     virtual void next() = 0;
5     virtual bool is_end() const = 0;
6     virtual Rid rid() const = 0;
7 };
8
9 class RmScan : public RecScan {
10 public:
11     RmScan(const RmFileHandle *file_handle);
12     void next() override;
13     bool is_end() const override;
14     Rid rid() const override;
15 };
```

(1) `RmScan(const RmFileHandle *file_handle);`

传入 `file_handle`，初始化 `rid`。

`RmScan` 内部存放了 `rid`，用于指向一个记录。

(2) `void next() override;`

用于找到文件中下一个存放了记录的位置。

对于当前页面，可以用bitmap来找bit为1的slot_no。如果当前页面的所有slot都没有存放record，就找下一个页面。

(3) `bool is_end() const override;`

判断是否到达文件末尾，即最后一个页面的最后一个slot。

可以自定义末尾的标识符，如 `RM_NO_PAGE`。

(4) `Rid rid() const override;`

获取 `RmScan` 内部存放的 `rid`。

实验计分

在本实验中，每个任务对应一个单元测试文件，每个测试文件中包含若干测试点。通过测试点即可得分，满分为100分。

测试文件及测试点如下：

任务点	测试文件	分值
任务1.1 磁盘存储管理器	src/test/storage/disk_manager_test.cpp	10
任务1.2 缓冲池替换策略	src/test/storage/lru_replacer_test.cpp	20
任务1.3 缓冲池管理器	src/test/storage/buffer_pool_manager_test.cpp	40
任务2 记录管理器	src/test/storage/record_manager_test.cpp	30